

Nanoexplanations

the blog of Aaron Sterling

Polygon rectangulation, part 3: Minimum-length rectangulation

Posted on [December 16, 2011](#) | [Leave a comment](#)

In this third (and final) post on polygon rectangulation, I will consider how to find the rectangulation of minimum total length for an orthogonal polygon. In [part one](#) of this short series, we considered rectangulations with a minimum number of rectangles; and, in [part two](#), we considered rectangulations with a minimum number of “fat” rectangles. I’ve saved this post for last, because this may be the most useful rectangulation application in VLSI, and this is the rectangulation problem that Ming-Yang Kao and I have applied to self-assembly (though I won’t discuss our application in this post).

The minimum-length rectangulation algorithm appeared in [Minimum Edge Length Partitioning of Rectilinear Polygons](#), by Lingas, Pinter, Rivest and Shamir (1982). The authors proved both a positive and a negative result. The positive result — which I will focus on today — is a $O(n^4)$ dynamic programming algorithm that finds an optimal minimum-length rectangulation for any orthogonal polygon *with no interior holes*. The negative result is a proof that, if the input polygon is allowed to have holes, then the problem is NP-complete. (I discussed the proof of this result in [a previous blog post](#).)

Optimal polygon triangulation

The dynamic programming algorithm for polygon rectangulation is similar in structure to a well-known one for polygon triangulation — finding a minimum-length set of nonoverlapping triangles that partitions the input polygon P . One online source for the triangulation algorithm is [these lecture notes](#) by David Eppstein. As far as we know, rectangulation is more complicated to solve than triangulation. (Among other things, minimum-length triangulation runs in $O(n^3)$, while the Lingas et al. algorithm runs in $O(n^4)$.) In the words of Lingas et al., “The difficult proof for the rectangular case involves showing how a given polygon can be split in a small number of ways while guaranteeing that the optimal partitioning is consistent with one of these splits.”

In brief, the triangulation problem is “simple,” because finding a triangulation of any input polygon reduces to finding a triangulation of a regular polygon. (Eppstein’s lecture notes explain why.) Since we can limit ourselves to input polygons with such nice features, the algorithm analysis is fairly simple. In the case of rectangulation, there is no known reduction to a simple class of input polygons. Instead, Lingas et al. show that only certain kinds of subfigures need to be considered, and that an optimal rectangulation of the entire polygon can be built up from optimal rectangulations of those subfigures. This makes the problem solvable via dynamic programming. We turn now to the method of rectangulation itself.

Proof of a key lemma

A key lemma in the MELPRP paper is:

Lemma 1. In all minimum edge length solutions for any given polygon P , all edges and all internal corners lie on the vertex grid.

As defined in the previous blog posts in this series, the vertex grid is the set of all points that lie on both vertical and horizontal lines that pass through a vertex of the input polygon. This is a very useful fact, since it limits the number of possible subrectangulations we need to evaluate. However, Lingas et al. assert it without proof. (Their 11-page paper appeared in a conference, and, to my knowledge, no journal version of the paper has ever appeared.) Perhaps this lemma is “obvious,” or folklore, but Ming-Yang Kao and I needed (a “slightly stronger” but still equivalent statement) for work we were doing, so we reproved lemma 1. The proof is short enough that I will reproduce it here.

Lemma. Let P be a polygon with all vertices lying on integer coordinates, and let R be a (finite) rectangulation of P . There is an algorithm that runs in time polynomial in the number of cuts of R , that produces a rectangulation R^ of P such that (1) all points of R^* lie on integer points, and (2) the total length of R^* is at most the total length of R .*

Proof. If all vertical cuts have integer x-coordinates, and all horizontal cuts have integer y-coordinates, then the rectangulation is an integer rectangulation, and we are done. Call the cuts that fail to have those properties *bad cuts*.

Claim: For each horizontal bad cut, we can translate it north or south, so that it will either encounter another horizontal bad cut, or reach an integer y-coordinate, without increasing the total perimeter of the rectangulation.

Proof of Claim: Let h be the horizontal bad cut we wish to translate. Let $v_1, \dots, v_k$ be all vertical cuts incident with h from the south. Let $v'_1, \dots, v'_l$ be all vertical cuts incident with h from the north. Suppose WLOG that $k \leq l$. We translate h north, extending each v_i so it remains incident with h , and shortening each v'_i so it remains incident with h as well. (If $k > l$ then we translate h south, performing the same lengthening and shortening operations, but switching the roles of the v_i and the v'_i .) Since we are shortening the v'_i by the same length as we are lengthening the v_i , we do not increase the total perimeter of the rectangulation. We continue translating h to the north until we encounter either another horizontal cut, or until the y -coordinate of h is an integer.

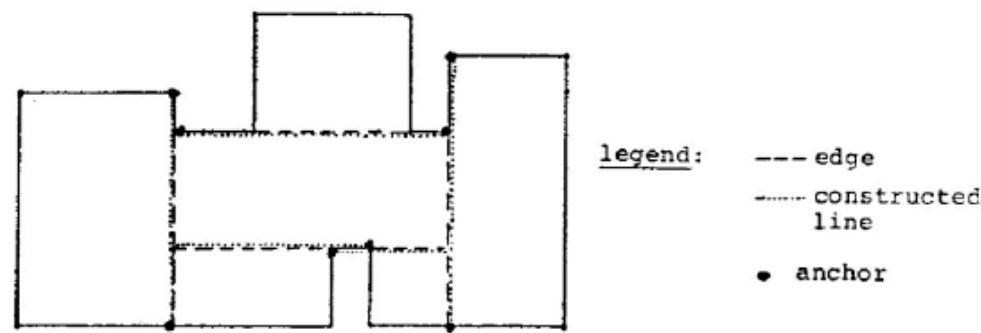
We run the following algorithm with input R . Initialize $R' = R$. Order the horizontal bad cuts, and order the vertical bad cuts. Go through the horizontal bad cuts one at a time. Let h be the horizontal bad cut under consideration. Translate it, without increasing the perimeter of R , until it has an integer y-coordinate, or until it encounters another horizontal cut. If h encounters another horizontal cut, call it h' , then remove both h and h' from R^* (and from the ordering of horizontal cuts of R), and place a new cut h'' into R^* , where the west endpoint of h'' is (min[x -coordinate of h , x-coordinate of h'], y -coordinate of h'), and the east endpoint of h'' is max[x-coordinate of h , x-coordinate of h'], y -coordinate of h'). (We name this process *absorption*, and say that h and h' are *absorbed* into h'' .) If h'' is at an integer y-coordinate, then we are done with this stage of the loop and we go on to the next horizontal cut in the order. Otherwise, it may be that we need to translate h'' either in the same, or the opposite direction as we were translating h previously. We continue translating in a non-lengthening direction, and absorbing if necessary, until we translate the horizontal cut at an integer y-coordinate. (There are at most number-of-cuts-many absorptions, so this process terminates efficiently.) Then we continue with the next horizontal cut in the ordering, until all are considered or removed from the ordering.

Once the bad horizontal cuts are all dealt with as above, we perform the same operations on the ordering of bad vertical cuts, except that we translate east-and-west instead of north-and-south. By the same arguments as above, in time polynomial in the number of vertical cuts, we place and/or absorb all vertical cuts onto integer x-coordinates without increasing the length of the rectangulation. Once all cuts have been considered, we have an integer rectangulation R^* such that $p(R') \leq p(R)$.

Set of subfigures

Given the input polygon P , we will decompose P into subfigures, find the minimum-length rectangulation of each

subfigure, then build from that a minimum-length rectangulation of P itself. Naively, we might expect that we have to consider every subfigure whose vertices lie on the vertex grid of P , and whose boundary is contained within the boundary of P . With a bit more analysis, it is possible to significantly reduce the set of subfigures that we need to consider. In the words of Lingas et al, let us call a *constructed line* “a maximal extension of a partitioning edge to include any sides of the boundary that are contiguous to the edge and go in the same direction. Two or more aligned edges may be put on the same constructed line, but this is not always the case.” Lingas et al. provide the following diagram as an example of constructed lines.



We are now able to state the key fact used to limit the size of the set of subfigures that must be considered by the rectangulation algorithm.

Fact. It suffices to consider subfigures whose boundary consists of a contiguous piece of the original boundary and at most two constructed lines (and the constructed lines are contiguous).

The partitioning rule

Fix a subfigure F that satisfies the previous Fact. We assume for the moment that we have already considered all (Fact-satisfying) subfigures that are contained inside F . We choose a *candidate point* for F by the following method.

1. If F has 0 constructed lines, choose any vertex of F .
2. If F has 1 constructed line, choose either endpoint of the constructed line.
3. If F has 2 constructed lines, choose the point where the constructed lines meet.

Once we have chosen the candidate point p_F , we consider each point on the vertex grid to be a possible *matching point*. A matching point q is one that defines a rectangle when paired with p_F — i.e., they are the opposite corners of a rectangle. There are at most $O(n^2)$ possible matching points inside any subfigure, and we only need to consider the points such that the induced rectangle lies entirely within F . (There are additional optimizations available also. For example, if p_F is a concave vertex, then its matching point must have the same x- or y-coordinate at p_F . I will leave this optimizat on to one side, though.)

The process so far has produced the following: a set of subfigures; a set of candidate points, one for each subfigure; and a set of matching points for each subfigure, at most $O(n^2)$ for each subfigure. We will now combine these ingredients into a dynamic programming algorithm for minimum-length rectangulation.

Dynamic programming algorithm

Given input polygon P , do the following

1. Produce all subfigures that satisfy the Fact above, and order them by area, from smallest to largest.
2. Find the candidate point for each figure.
3. Loop through the figures, in order.

- A. When considering figure F , loop through the set of matching points $m_1, m_2, \dots$.
 - i. Calculate the length of the rectangulation of F obtained by drawing a rectangle R defined by the candidate point and the matching point m_i . This can be done quickly because previously in the order of figures we computed the minimum length of the figure F' , which is F minus the rectangle R .
 - B. Find the minimum over all rectangulations obtained in Step A(i). Store that as the minimum-length rectangulation for figure F .
4. Halt when we have computed the value of the maximum figure, that is, the minimum-length rectangulation of P .

Since there $O(n^2)$ figures that satisfy the Fact, and, for each figure, there are $O(n^2)$ matching points, the total time of the algorithm is $O(n^4)$.

Andrzej Lingas, Ron Y. Pinter, Ronald R. Rivest, & Adi Shamir (1982). Minimum edge length partitioning of rectilinear polygons *Proceedings of the 20th Allerton Conference on Communication*, 53-63

This entry was posted in [Uncategorized](#) and tagged [computational geometry](#), [orthogonal polygon](#), [rectangular partitioning](#), [rectilinear polygon](#). Bookmark the [permalink](#).